

SIMATS ENGINEERING

CO5-ASSESSMENT TOOL1-Design Task

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

Register Number	192511426
Name	K.B.Devarshan

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 40%

Time Duration: 1 Hour

QUESTIONS: 5

Total Marks:50

Code of Conduct:

I_(K.B.Devarshan/192511426)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



1. End-to-End System Design

Design a scalable AI-based system using **PySpark and RDD operations** to analyze real-time social media data and generate intelligent recommendations, including system architecture, data flow, and agent integration.

2. Distributed Intelligent Agent System

Develop a **multi-agent AI system architecture** that uses PySpark for distributed data processing and enables agents to collaborate in solving a large-scale optimization problem.

3. Big Data AI Pipeline Design

Design a complete **distributed AI pipeline** using RDD transformations for data preprocessing, feature engineering, and model training, ensuring scalability and fault tolerance.

4. Real-Time Decision-Making System

Construct an AI-based system integrating **RDD-based streaming data processing** with intelligent agents for real-time decision-making in applications such as smart traffic or healthcare.

5. Cloud-Based Distributed AI Application

Design a **cloud-enabled AI system** using PySpark and intelligent agent architectures that supports large-scale data analytics, dynamic resource allocation, and adaptive learning.

RUBRICS FOR CALCULATION

Evaluation Criteria	Problem Understanding & Requirement Analysis	System Architecture Design	Use of PySpark & RDD Operations	Intelligent Agent Integration	Scalability & Distributed Computing Design	Data Flow & Processing Pipeline	Innovation & Creativity	Clarity, Presentation & Documentation
Weightage	10%	20%	15%	15%	10%	10%	10%	10%

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformations/actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration
Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed
Data Flow & Processing Pipeline	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined

Innovation & Creativity	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality
Clarity, Presentation & Documentation	Very clear, neat diagrams, well-organized report	Mostly clear	Somewhat organized	Poor presentation

Question 1 – PySpark RDD-Based Social Media Recommendation System

Aim

To design and implement a simple scalable AI-based system using **PySpark and RDD operations** to analyze social media data, identify trending topics, and generate intelligent recommendations using an agent.

Problem Statement

Social media platforms continuously generate large amounts of user-generated data. Manually analyzing these posts to identify popular topics is difficult. The proposed system uses **PySpark RDD operations** to process social media posts, count frequently occurring topics, identify trends, and provide recommendations through a simple intelligent agent.

Objective

The main objectives of the system are:

1. To create and process social media data using PySpark RDDs.
2. To apply RDD operations such as filter(), flatMap(), map(), and reduceByKey().
3. To identify frequently occurring and trending topics.
4. To integrate a simple recommendation agent.
5. To demonstrate how the system can be scaled for large amounts of social media data.

Algorithm:

Step 1 – Create Spark Context

Step 2 – Create RDD

Step 3 – Filter Data

Step 4 – Extract Words

Step 5 – Create Key-Value Pairs

Step 6 – Count Words

Step 7 – Remove Common Words

Step 8 – Identify Trending Topics

Step 9 – Recommendation Agent

Program:

```
import os
import sys

os.environ["PYSPARK_PYTHON"] = sys.executable
os.environ["PYSPARK_DRIVER_PYTHON"] = sys.executable

from pyspark import SparkConf
from pyspark import SparkContext

conf = SparkConf().setAppName("SocialMediaAI").setMaster("local[*]")
conf.set("spark.driver.host", "127.0.0.1")
conf.set("spark.driver.bindAddress", "127.0.0.1")

sc = SparkContext(conf=conf)

posts = [
    "AI is amazing",
    "AI is useful",
    "Python is powerful",
    "AI is trending",
    "Python is easy"
]

posts_rdd = sc.parallelize(posts)

clean_rdd = posts_rdd.filter(lambda x: len(x.strip()) > 0)

words_rdd = clean_rdd.flatMap(lambda x: x.lower().split())

word_pairs = words_rdd.map(lambda word: (word, 1))

word_count = word_pairs.reduceByKey(lambda a, b: a + b)

stop_words = {"is", "the", "a", "an"}

filtered_words = word_count.filter(
    lambda x: x[0] not in stop_words
)

trending = filtered_words.sortBy(
    lambda x: x[1],
    ascending=False
)

top_topics = trending.take(3)

print("===== TRENDING TOPICS =====")

for word, count in top_topics:
    print(word, "->", count)
```

```

def recommendation_agent(topics):
    top_word, count = topics[0]

    if count >= 3:
        return f"{top_word} is highly trending. Recommend more {top_word}-related content."
    elif count >= 2:
        return f"{top_word} is moderately trending. Recommend some {top_word}-related content."
    else:
        return f"{top_word} has low activity. Continue monitoring."

print("\n===== RECOMMENDATION =====")
print(recommendation_agent(top_topics))

sc.stop()

```

Input:

A small collection of sample social media posts is used as input:

AI is amazing
 AI is useful
 Python is powerful
 AI is trending
 Python is easy

RDD Operations Used

RDD Operation	Purpose
parallelize()	Creates an RDD from the input data
filter()	Removes unwanted data
flatMap()	Splits posts into individual words
map()	Creates (word, 1) pairs
reduceByKey()	Counts the occurrence of each word
sortBy()	Sorts topics according to frequency
take()	Selects the top results

Scalability

The system can be scaled to handle large volumes of social media data because Spark can divide RDD data into partitions and process them in parallel across multiple worker nodes. RDD lineage also provides fault tolerance by allowing lost partitions to be recomputed.

System Architecture

1. Social Media Posts
2. SparkContext
3. RDD
4. filter()
5. flatMap()
6. map()
7. reduceByKey()
8. Trending Topics
9. Recommendation Agent
10. Recommendation

Agent Integration

The system uses a simple **Recommendation Agent**.

The agent follows:

Observe – Analyze-- Decide –Recommend

The social media posts act as the input observed by the agent. The RDD processing provides the agent with information about topic frequency. The agent then identifies the most popular topic and generates an appropriate recommendation.

Output:

```
🟡 Reading from archive can take up to a minute...
🟢 Installing Modules...
🟢 Machine Ready

Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
===== TRENDING TOPICS =====
ai -> 3
python -> 2
amazing -> 1

===== RECOMMENDATION =====
ai is highly trending. Recommend more ai-related content.
```

Result

The PySpark-based social media analysis system successfully processes the input posts using RDD operations, identifies the most frequent topics, and generates an intelligent recommendation based on the detected trend.

Question 2 – Distributed Intelligent Agent System

Aim To design and implement a multi-agent AI system architecture using PySpark for distributed data processing, enabling agents to collaborate in solving a large-scale delivery optimization problem.

Problem Statement Logistics companies face challenges when assigning thousands of delivery tasks to multiple drivers across different zones. Centralized calculation becomes too slow as data grows. The proposed system uses PySpark RDDs to distribute the delivery data and utilizes intelligent agents to evaluate workloads, optimizing task allocation across the network.

Objective

- To process large-scale geographic task data using PySpark RDDs.
- To group tasks by delivery zones using `map()` and `groupByKey()`.
- To evaluate zone workloads to find optimal driver assignments.
- To integrate collaborative intelligent agents that balance the delivery load.
- To ensure the architecture scales across multiple cluster nodes for large optimization problems.

Algorithm

- Step 1 – Create Spark Context
- Step 2 – Load Delivery Tasks into RDD
- Step 3 – Map Tasks to Delivery Zones
- Step 4 – Group Tasks by Zone
- Step 5 – Calculate Workload per Zone
- Step 6 – Multi-Agent Optimization (Assign Drivers)

Program

Python

```
import os, sys
os.environ["PYSPARK_PYTHON"] = sys.executable
os.environ["PYSPARK_DRIVER_PYTHON"] = sys.executable

from pyspark import SparkConf, SparkContext

conf = SparkConf().setAppName("MultiAgentOptimization").setMaster("local[*]")
sc = SparkContext(conf=conf)

# Format: (Task_ID, Zone, Weight_kg)
tasks = [
    ("T1", "North", 10), ("T2", "South", 5),
    ("T3", "North", 15), ("T4", "East", 8),
    ("T5", "South", 12), ("T6", "North", 5)
]

tasks_rdd = sc.parallelize(tasks)

# Create key-value pairs (Zone, Weight)
zone_pairs = tasks_rdd.map(lambda x: (x[1], x[2]))

# Calculate total weight per zone
zone_workload = zone_pairs.reduceByKey(lambda a, b: a + b)

workloads = zone_workload.collect()
```

```

print("==== ZONE WORKLOADS ====")
for zone, load in workloads:
    print(f'Zone {zone}: {load} kg')

# Intelligent Agent Logic
def allocation_agent(workload_data):
    allocations = []
    for zone, load in workload_data:
        drivers_needed = (load // 10) + 1 # 1 driver per 10kg
        allocations.append(f'Agent assigned {drivers_needed} driver(s) to {zone} zone.')
    return allocations

print("\n==== AGENT ALLOCATION ====")
decisions = allocation_agent(workloads)
for d in decisions:
    print(d)

sc.stop()

```

Input

A collection of delivery tasks specifying the task ID, geographic zone, and package weight.

RDD Operations Used

RDD Operation	Purpose
parallelize()	Creates an RDD from the distributed task data.
map()	Extracts the Zone as the key and Weight as the value.
reduceByKey()	Aggregates the total weight workload for each specific zone.
collect()	Retrieves the aggregated workload data to the driver node for the agent.

Scalability This architecture easily scales for nationwide logistics. As millions of tasks stream in, PySpark partitions the data geographically across worker nodes. The distributed data processing ensures that agents only receive summarized workload metrics, preventing processing bottlenecks.

System Architecture

1. Raw Task Data
2. SparkContext
3. RDD Creation
4. map() (Extract Zone/Weight)
5. reduceByKey() (Aggregate Load)
6. Dispatch Agent
7. Optimized Driver Allocation

Agent Integration The system uses a Multi-Agent architecture for task allocation. The **Observer Agent** processes the RDD outputs to understand the current weight in each zone. The **Allocator Agent** then applies predefined optimization logic (e.g., driver capacity constraints) to dynamically allocate the required workforce to the zones experiencing the highest demand.

Output

```
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
===== ZONE WORKLOADS =====
Zone South: 17 kg
Zone East: 8 kg
Zone North: 30 kg

===== AGENT ALLOCATION =====
Agent assigned 2 driver(s) to South zone.
Agent assigned 1 driver(s) to East zone.
Agent assigned 4 driver(s) to North zone.

Execution by Demo Interviewer finished in 7.49s
```

Result : The PySpark multi-agent system successfully distributed the raw logistics data, calculated the regional workload through RDD transformations, and utilized an intelligent agent to optimize and allocate the appropriate number of drivers per zone.

Question 3 – Big Data AI Pipeline Design

Aim To design a complete distributed AI pipeline using RDD transformations for data preprocessing, feature engineering, and basic model evaluation, ensuring scalability and fault tolerance.

Problem Statement

Training AI models on massive, raw datasets often results in bottlenecks because data cleaning and feature extraction overwhelm single machines. The proposed system utilizes PySpark RDDs to distribute the preprocessing pipeline—cleaning data and engineering features in parallel—before passing the structured data to an intelligent agent for evaluation.

Objective

- To construct a robust data pipeline using PySpark RDDs.
- To perform distributed data preprocessing (filtering missing values).
- To apply feature engineering (calculating ratios or scaling data) using `map()`.
- To aggregate training metrics using RDD actions.
- To guarantee scalability and fault tolerance during model preparation.

Algorithm

- Step 1 – Create Spark Context
- Step 2 – Load Raw Training Data (RDD)
- Step 3 – Preprocess (Filter invalid/missing records)
- Step 4 – Feature Engineering (Extract and calculate new metrics)
- Step 5 – Aggregate Data for Model Training
- Step 6 – Pipeline Agent Evaluation

Program

Python

```
import os
```

```
import sys
```

```
os.environ["PYSPARK_PYTHON"] = sys.executable
```

```
os.environ["PYSPARK_DRIVER_PYTHON"] = sys.executable
```

```
from pyspark import SparkConf, SparkContext
```

```
conf = SparkConf().setAppName("BigDataAIPipeline").setMaster("local[*]")
```

```
sc = SparkContext(conf=conf)
```

```
# Raw Data: (House_ID, Square_Feet, Price_in_Thousands)
```

```
# 0 indicates missing/corrupt data
```

```
raw_data = [
```

```
    ("H1", 1000, 150),
```

```
    ("H2", 1500, 225),
```

```
    ("H3", 0, 300),
```

```
    ("H4", 2000, 300),
```

```
    ("H5", 1200, 0)
```

```
]
```

```
raw_rdd = sc.parallelize(raw_data)
```

```
# Preprocessing: Filter out corrupt records (0 values)
```

```
clean_rdd = raw_rdd.filter(lambda x: x[1] > 0 and x[2] > 0)
```

```
# Feature Engineering: Calculate Price per Square Foot
```

```
# Map to (House_ID, Price_per_SqFt)
```

```
features_rdd = clean_rdd.map(lambda x: (x[0], round((x[2] * 1000) / x[1], 2)))
```

```
processed_data = features_rdd.collect()
```

```
print("===== ENGINEERED FEATURES =====")
```

```
for house, price_sqft in processed_data:
```

```
    print(f'House {house}: ${price_sqft} per sq ft')
```

```
# Aggregate for a basic model insight (Average Price per SqFt)
```

```
total_price_sqft = features_rdd.map(lambda x: x[1]).sum()
```

```
count = features_rdd.count()
```

```
avg_price_sqft = total_price_sqft / count
```

```
def pipeline_agent(avg_metric, processed_count):
```

```
    if processed_count < 3:
```

```
        return "Warning: Not enough valid data points for reliable training."
```

```
    return f'Pipeline Agent: Ready for training. Market average is ${round(avg_metric, 2)} per sq ft based on {processed_count} valid records.'
```

```
print("\n===== AGENT EVALUATION =====")
```

```
print(pipeline_agent(avg_price_sqft, count))
```

```
sc.stop()
```

Input

A dataset containing raw housing information (ID, Size, Price), intentionally including some corrupt or missing records (zeroes) to demonstrate the cleaning pipeline.

RDD Operations Used

RDD Operation	Purpose
parallelize()	Loads the raw training data into a distributed RDD.
filter()	Preprocesses the data by removing incomplete or invalid rows.
map()	Performs feature engineering by transforming raw numbers into a new metric.
sum() & count()	Actions used to aggregate the final pipeline statistics.

Scalability By chaining RDD transformations (filter -> map), Spark achieves lazy evaluation. This means the engine optimizes the entire execution plan before running it. If a node fails during feature engineering, Spark's fault tolerance ensures the lineage graph automatically recomputes only the lost partition, maintaining a highly reliable pipeline.

System Architecture

1. Raw Dataset
2. SparkContext
3. filter() (Data Preprocessing)
4. map() (Feature Engineering)
5. Statistical Aggregation
6. Pipeline Agent Validation
7. Ready-for-Training Output

Agent Integration The Pipeline Agent acts as a quality assurance gatekeeper. It evaluates the output of the distributed data processing to ensure enough high-quality data survived the preprocessing phase before allowing an AI model to begin the expensive training process.

Output

```
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
===== ENGINEERED FEATURES =====
House H1: $150.0 per sq ft
House H2: $150.0 per sq ft
House H4: $150.0 per sq ft

===== AGENT EVALUATION =====
Pipeline Agent: Ready for training. Market average is $150.0 per sq ft based on 3 valid records.
```

Result The Big Data AI Pipeline successfully demonstrated data preprocessing and feature engineering using PySpark. The RDD transformations cleaned the data efficiently, and the integrated agent verified the integrity of the output, proving the system's readiness for large-scale AI model training.

Question 4 – Real-Time Decision-Making System

Aim To construct an AI-based system integrating PySpark RDD processing with intelligent agents for real-time decision-making in a smart traffic management application.

Problem Statement Urban areas suffer from severe traffic congestion because static traffic light timers fail to adapt to real-time road conditions. The proposed system uses PySpark to rapidly process streaming vehicle counts from distributed IoT sensors, allowing an intelligent agent to dynamically adjust traffic light durations and optimize vehicle flow.

Objective

- To process real-time traffic sensor data using PySpark RDDs.
- To clean anomalous sensor readings using the filter() operation.
- To compute total vehicle density per junction using map() and reduceByKey().
- To integrate a real-time decision-making agent that controls signal timings.
- To demonstrate system scalability capable of handling rush-hour data spikes.

Algorithm

- Step 1 – Initialize Spark Context
- Step 2 – Ingest Real-Time Traffic Data
- Step 3 – Filter Invalid Sensor Readings
- Step 4 – Map Data to Junction-Vehicle Pairs
- Step 5 – Aggregate Total Traffic per Junction
- Step 6 – Agent Evaluates Density and Adjusts Signal Timing

Program

Python

```
import os
```

```
import sys
```

```
from pyspark import SparkConf, SparkContext
```

```
os.environ["PYSPARK_PYTHON"] = sys.executable
```

```
os.environ["PYSPARK_DRIVER_PYTHON"] = sys.executable
```

```
conf = SparkConf().setAppName("SmartTrafficAgent").setMaster("local[*]")
```

```
# Safely stop any existing SparkContext
```

```
try:
```

```
SparkContext.getOrCreate().stop()
```

```
except:
```

```
    pass
```

```
sc = SparkContext(conf=conf)
```

```
# Format: (Junction_ID, Sensor_ID, Vehicle_Count)
```

```
# -1 simulates a sensor error
```

```
traffic_data = [  
    ("J1", "S1", 45), ("J1", "S2", 55),  
    ("J2", "S1", 10), ("J2", "S2", 5),  
    ("J3", "S1", 80), ("J3", "S2", 70),  
    ("J1", "S3", -1)  
]
```

```
traffic_rdd = sc.parallelize(traffic_data)
```

```
# Filter out negative/error readings
```

```
clean_rdd = traffic_rdd.filter(lambda x: x[2] >= 0)
```

```
# Map to (Junction_ID, Vehicle_Count)
```

```
junction_pairs = clean_rdd.map(lambda x: (x[0], x[2]))
```

```
# Calculate total vehicles per junction
```

```
total_traffic = junction_pairs.reduceByKey(lambda a, b: a + b)
```

```
traffic_status = total_traffic.collect()
```

```
print("==== CURRENT TRAFFIC DENSITY ====")
```

```
for junction, count in traffic_status:
```

```
    print(f'Junction {junction}: {count} vehicles')
```

```
# Intelligent Decision-Making Agent
```

```
def traffic_agent(traffic_status_data):
```

```
    decisions = []
```

```
    for junction, count in traffic_status_data:
```

```
        if count > 100:
```

```
            decisions.append(f'ALERT at {junction}: Heavy traffic. Set GREEN light to 90 seconds.')
```

```
        elif count > 40:
```

```
            decisions.append(f'Moderate traffic at {junction}. Set GREEN light to 45 seconds.')
```

```
        else:
```

```
            decisions.append(f'Low traffic at {junction}. Set GREEN light to 20 seconds.')
```

```
    return decisions
```

```
print("\n==== AGENT DECISIONS ====")
```

```
actions = traffic_agent(traffic_status)
```

```
for action in actions:
```

```
    print(action)
```

```
sc.stop()
```

Input

Simulated IoT sensor readings containing the Junction ID, Sensor ID, and the current vehicle count detected in that lane.

RDD Operations Used

RDD Operation	Purpose
parallelize()	Simulates the intake of distributed sensor data streams.
filter()	Removes anomalous or broken sensor readings (e.g., negative counts).
map()	Isolates the Junction ID as the key and the vehicle count as the value.
reduceByKey()	Computes the aggregated traffic density per junction in real-time.

Scalability PySpark is designed to distribute data processing across worker nodes. As a city expands and thousands of new sensors are added to the grid, the RDD architecture partitions the incoming data geographically, ensuring that sudden spikes in data during rush hour do not create bottlenecks or crash the decision engine.

System Architecture

1. IoT Traffic Sensors
2. SparkContext Data Ingestion
3. filter() (Data Cleaning)
4. map() & reduceByKey() (Density Calculation)
5. Traffic State Evaluation
6. Decision-Making Agent
7. Signal Output Adjustment

Agent Integration An intelligent Traffic Agent operates as the primary decision-maker. It *observes* the aggregated vehicle counts outputted by PySpark, *decides* on the severity of the congestion based on predefined capacity thresholds, and *acts* by adjusting the green light durations to instantly alleviate road bottlenecks.

Output

```
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
===== CURRENT TRAFFIC DENSITY =====
Junction J1: 100 vehicles
Junction J3: 150 vehicles
Junction J2: 15 vehicles

===== AGENT DECISIONS =====
Moderate traffic at J1. Set GREEN light to 45 seconds.
ALERT at J3: Heavy traffic. Set GREEN light to 90 seconds.
Low traffic at J2. Set GREEN light to 20 seconds.

Execution by Demo Interviewer finished in 8.34s
```

Result The AI system successfully processed distributed traffic sensor data using robust RDD operations and dynamically formulated real-time signal control decisions through an integrated intelligent agent.

Question 5 – Cloud-Based Distributed AI Application

Aim To design a cloud-enabled AI system using PySpark and intelligent agent architectures that supports large-scale data analytics, dynamic resource allocation, and adaptive learning.

Problem Statement Cloud computing platforms constantly face fluctuating user demands and workload spikes. Allocating static server resources leads to either expensive idle time or severe system slowdowns. The proposed system utilizes PySpark to analyze large-scale cluster usage data and employs an intelligent agent to dynamically allocate resources (like Virtual Machines) while adaptively learning threshold limits based on historical loads.

Objective

- To process large-scale server usage logs using PySpark RDDs.
- To calculate the total processing load per cloud cluster using map() and reduceByKey().
- To implement dynamic resource allocation via an intelligent agent.
- To introduce adaptive learning by having the agent adjust its scaling thresholds over time.
- To demonstrate cloud scalability and efficient data analytics.

Algorithm

- Step 1 – Initialize Spark Context for Cloud
- Step 2 – Load Server Workload Data (RDD)
- Step 3 – Map Data to Cluster-Load Pairs
- Step 4 – Aggregate Total CPU Load per Cluster
- Step 5 – Agent Evaluates Load against Adaptive Thresholds
- Step 6 – Agent Executes Dynamic Resource Allocation

Program

Python

```
import os
```

```
import sys
```

```
from pyspark import SparkConf, SparkContext
```

```
os.environ["PYSPARK_PYTHON"] = sys.executable
```

```
os.environ["PYSPARK_DRIVER_PYTHON"] = sys.executable
```

```
conf = SparkConf().setAppName("CloudResourceAgent").setMaster("local[*]")
```

```
# Safely stop any existing SparkContext
```

```
try:
```

```
SparkContext.getOrCreate().stop()
```

```
except:
```

```
    pass
```

```
sc = SparkContext(conf=conf)
```

```
# Format: (Cluster_ID, Server_ID, CPU_Load_Percentage)
```

```
cloud_logs = [
```

```
    ("Cluster_A", "VM1", 85), ("Cluster_A", "VM2", 90),
```

```
    ("Cluster_B", "VM1", 20), ("Cluster_B", "VM2", 30),
```

```
    ("Cluster_C", "VM1", 95), ("Cluster_C", "VM2", 98),
```

```
    ("Cluster_A", "VM3", 88)
```

```
]
```

```
logs_rdd = sc.parallelize(cloud_logs)
```

```
# Map to (Cluster_ID, CPU_Load)
```

```
cluster_pairs = logs_rdd.map(lambda x: (x[0], x[2]))
```

```
# Calculate total CPU load and count per cluster to find the average
```

```
# mapValues transforms (CPU_Load) -> (CPU_Load, 1) for counting
```

```
load_counts = cluster_pairs.mapValues(lambda v: (v, 1))
```

```
# reduceByKey sums up both CPU_Loads and the counts
```

```
total_load_counts = load_counts.reduceByKey(lambda a, b: (a[0] + b[0], a[1] + b[1]))
```

```
# Calculate the average load per cluster
```

```
avg_cluster_load = total_load_counts.mapValues(lambda v: v[0] / v[1]).collect()
```

```
print("==== CLUSTER LOAD ANALYTICS ====")
```

```
for cluster, avg_load in avg_cluster_load:
```

```
    print(f'{cluster}: {round(avg_load, 2)}% average CPU load')
```

```
# Intelligent Agent with Adaptive Learning Simulation
```

```
def cloud_scaling_agent(cluster_data, base_threshold=80):
```

```
    actions = []
```

```
    # Simulating adaptive learning: Threshold drops if overall system is stressed
```

```
    system_stress = sum(load for _, load in cluster_data) / len(cluster_data)
```

```
    dynamic_threshold = base_threshold - 5 if system_stress > 70 else base_threshold
```

```
    actions.append(f'Agent adapted scaling threshold to: {dynamic_threshold}%\n')
```

```
    for cluster, avg_load in cluster_data:
```

```
        if avg_load > dynamic_threshold:
```

```
            actions.append(f'SCALE UP: {cluster} exceeded threshold. Provisioning 2 new VMs.')
```

```
        elif avg_load < 40:
```

```
            actions.append(f'SCALE DOWN: {cluster} is underutilized. Decommissioning 1 VM.')
```

```
        else:
```

```
            actions.append(f'STABLE: {cluster} load is optimal. No action required.')
```

```
    return actions
```

```
print("\n==== AGENT RESOURCE ALLOCATION ====")
```

```
scaling_decisions = cloud_scaling_agent(avg_cluster_load)
```

for decision in scaling_decisions:

```
print(decision)
```

```
sc.stop()
```

Input

Simulated server logs from a cloud environment containing the Cluster ID, Virtual Machine (VM) ID, and the current CPU Load Percentage.

RDD Operations Used

RDD Operation	Purpose
parallelize()	Loads distributed server logs into PySpark.
mapValues()	Prepares the data for averaging by appending a count metric.
reduceByKey()	Aggregates the massive volumes of server data to calculate regional loads.
collect()	Returns the processed analytics to the driver for the agent to act upon.

Scalability The architecture is inherently built for cloud environments. PySpark abstracts the data distribution, allowing the framework to rapidly calculate massive data lakes of server logs across hundreds of nodes. When the data analytics signal high stress, the scaling agent allocates new computing power instantly.

System Architecture

- 1. Distributed Cloud Logs
- 2. SparkContext
- 3. mapValues() (Data Transformation)
- 4. reduceByKey() (Large-Scale Analytics)
- 5. Adaptive Threshold Calculation
- 6. Cloud Scaling Agent
- 7. Dynamic VM Allocation/Deallocation

Agent Integration This system integrates a highly autonomous agent capable of dynamic resource allocation and adaptive learning. The agent reads the large-scale analytics outputted by the RDD operations, evaluates the overall system stress to adaptively adjust its own scaling thresholds, and finally executes precise virtual machine provisioning commands to balance the network.

Output

```
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
===== CLUSTER LOAD ANALYTICS =====
Cluster_A: 87.67% average CPU load
Cluster_B: 25.0% average CPU load
Cluster_C: 96.5% average CPU load

===== AGENT RESOURCE ALLOCATION =====
Agent adapted scaling threshold to: 80%

SCALE UP: Cluster_A exceeded threshold. Provisioning 2 new VMs.
SCALE DOWN: Cluster_B is underutilized. Decommissioning 1 VM.
SCALE UP: Cluster_C exceeded threshold. Provisioning 2 new VMs.
```

Result The cloud-based system effectively processed distributed load metrics using PySpark RDDs, while the integrated intelligent agent successfully demonstrated adaptive learning by shifting its threshold and dynamically allocating resources based on large-scale analytics.